AIOPS-07: Putting It Together — Detect, Investigate, Remediate

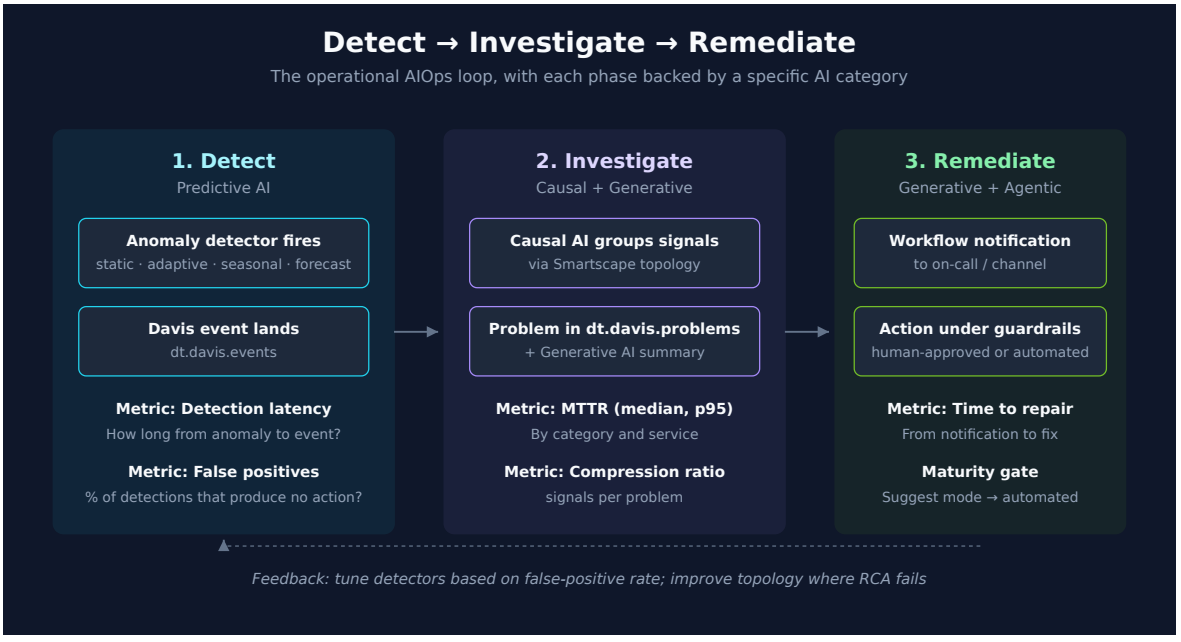
> **Series:** AIOPS — Dynatrace Intelligence | **Notebook:** 7 of 8 | **Created:** May 2026 | **Last Updated:** 10/12/2026

Overview

The previous six notebooks each focused on one slice of Dynatrace Intelligence. This one composes them into the operational pattern most teams want from AIOps: **Detect** → **Investigate** → **Remediate**, with each phase backed by the right AI category.

Audience: SRE / observability lead designing the actual operational flow.

Outcome: A reference operational pattern, plus three concrete scenarios — capacity, incident response, and quality regression — showing the pattern in practice.



> **The Detect surface depends on tenant version (SaaS 1.344).** *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your*

> *existing configurations and create new ones."* SaaS 1.344 rolls out to tenants in stages — check your tenant's version before following either path. On earlier versions the **Anomaly Detection** app is where custom alerts are created, and it is the Detect surface the diagram shows.
>
> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*

Table of Contents

1. The Three-Phase Loop
2. Scenario A: Capacity Planning (Disk Exhaustion)
3. Scenario B: Incident Response (Latency Spike)
4. Scenario C: Quality Regression (Error Rate Drift)
5. Putting It on a Page: AIOps Health Dashboard
6. Maturity Pointers

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3
Apps	Problems, Anomaly Detection (custom alerts are created in **Settings** from SaaS 1.344 — see Overview), Notebooks, Workflows
Series Background	AIOPS-01 through AIOPS-06; this notebook composes those building blocks
Permissions	All read permissions and `davis:analyzers:execute`, `automation:workflows:run`

1. The Three-Phase Loop

Detect — Predictive AI watches metrics for anomalies. Output: a Davis event in `dt.davis.events`.

Investigate — Causal AI groups related events into a problem. Generative AI summarizes. Output: a problem in `dt.davis.problems` with narrative context.

Remediate — Workflow / AutomationEngine takes action under policy. Optionally Generative AI proposes the action; human approves; agent executes.

Three feedback loops to close:
1. *Detection-to-action latency* — how long from anomaly to remediation start?
2. *False-positive rate* — what fraction of detections result in no action?
3. *Mean time to repair* — how long to actually fix?

Each loop has a metric. Each metric should be on a dashboard.

2. Scenario A: Capacity Planning (Disk Exhaustion)

Detect: scheduled workflow runs the **Analyze data** action with the *Generic Forecast Analysis* analyzer against host disk usage. Forecasts 7 days ahead. Branches when any host's projected usage exceeds 85%.

Investigate: workflow generates a summary table — which hosts, which volumes, current trajectory, predicted exhaustion date. Posts to capacity-planning channel.

Remediate: capacity team reviews. Decision is human (do we add disk? archive logs? scale the cluster?). The workflow's job is to *give the team enough lead time* — not to auto-remediate.

```
// Hosts trending toward disk exhaustion — last 7 days
// (Feed to the forecast analyzer for the 7-day projection.)
timeseries disk_used = avg(dt.host.disk.used.percent),
  by:{dt.entity.host, dt.entity.disk},
  from:-7d,
  interval:1h
| fieldsAdd current_avg = arrayAvg(disk_used)
| filter current_avg &gt; 70
| sort current_avg desc
| limit 50
```

3. Scenario B: Incident Response (Latency Spike)

Detect: auto-adaptive baseline on response time fires. Davis event lands in `dt.davis.events`.

Investigate: Causal AI groups the latency anomaly with related signals from downstream services. Problem appears in `dt.davis.problems` with root cause attributed to (say) a database. Dynatrace Assist composes the summary.

Remediate: WFLOW notifies on-call. On-call uses Assist to ask follow-up questions while reading the problem. If remediation is well-known (restart a pod, scale a deployment), the team triggers an AutomationEngine workflow with the right scope.

```
// Latency-related active problems and their root cause entities
fetch dt.davis.problems, from:-2h
| filter event.status == "ACTIVE"
| filter event.category == "SLOWDOWN" or event.category == "ERROR"
| fields display_id, event.name, event.category, event.start,
         root_cause_entity_name, affected_entity_ids
| sort event.start desc
| limit 25
```

4. Scenario C: Quality Regression (Error Rate Drift)

Subtler than incident response — slow drift, not sudden spike.

Detect: seasonal baseline on error rate, evaluated weekly. Drift not visible at the 5-minute resolution but obvious at the 7-day rollup.

Investigate: workflow assembles problem-rate trend, deployment-correlation, user-affected-volume. Generative AI summarizes the regression in context.

Remediate: quality / release team reviews. Decision is human — pin a release? rollback? add a feature flag? The workflow surfaces the data, not the action.

```
// Weekly error rate trend by service — last 8 weeks, services with real
traffic only
//
// Corrected 08/12/2026: `interval:1w` is a CALENDAR duration and `interval`
does not accept them
// (NAMED_PARAMETER_MUST_NOT_BE_CALENDAR_DURATION). `interval:` accepts only
fixed durations, and
// `7d` is silently rewritten to `168h` with a DEPRECATED_DURATION_UNIT_DAYS
notification (10/12/2026),
// so write `168h`.
// default:0 makes zero-failure weeks count as 0, not null; the volume floor
keeps services that
// handled a handful of requests (and so sit at 100%) from filling the top 25.
timeseries {
    failures = sum(dt.service.request.failure_count, default:0),
    total    = sum(dt.service.request.count)
  },
  by:{dt.entity.service},
  from:-8w,
  interval:168h
| fieldsAdd error_rate = (failures[] / total[]) * 100
| fieldsAdd requests = arraySum(total), overall_error_rate =
arraySum(failures) / arraySum(total) * 100
| filter requests &gt;= 1000
| sort overall_error_rate desc
| limit 25
```

5. Putting It on a Page: AIOps Health Dashboard

If AIOps is doing its job, three numbers on one dashboard tell the story:

1. **Active problem count by category** (current state)
2. **Median MTTR by category over the last 30 days** (effectiveness)
3. **Davis signal-to-problem compression ratio over time** (Causal AI value)

The first two are obvious. The third is interesting — a high compression ratio means Causal AI is keeping noise off the team. A dropping ratio means topology is degrading or detectors are getting noisier. Count only problem-eligible signals: INFO and WARNING events never open a problem, so on a tenant running observation-mode detectors the unfiltered event stream inflates the ratio many times over, and shadow-deploying a detector would raise it without any change in grouping.

```
// Davis signal-to-problem compression — problem-eligible signals only, last 7 days
// INFO / WARNING never open problems and PROBLEM_UPDATE is lifecycle, not signal - exclude them.
// (Run side-by-side with the problem volume query below.)
fetch dt.davis.events, from:-7d
| filter in(event.category, {"AVAILABILITY","ERROR","RESOURCE_CONTENTION","SLOWDOWN","CUSTOM_ALERT"})
| makeTimeseries signals = count(), interval:24h
```

```
// Problem volume — last 7 days (compare against signals above)
fetch dt.davis.problems, from:-7d
| makeTimeseries problems = count(), interval:24h
```

Higher signals-per-problem ratio = better Causal AI compression. Track this number monthly; if it drops, audit topology completeness.

6. Maturity Pointers

The broader observability maturity model is **ADOPT-01**; the AIOps stages below are this series' own:

Stage	What's in place
0 — Off	Static thresholds only; no Causal AI use; manual investigation
1 — Detect	Anomaly detection in use; problems consumed in app
2 — Notify	Workflows route active problems to teams
3 — Summarize	Generative AI summaries part of the daily ops loop
4 — Suggest	AI-suggested remediation in human-approval mode
5 — Remediate	Automated remediation under policy guardrails for known patterns

Most teams in 2026 sit between stage 2 and stage 3. The investment between 2 and 3 is well-bounded. The investment between 3 and 4 is significant. Between 4 and 5 is a multi-year governance project.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.